Optimization Against AI Slop: Three Empirical Workstreams from the PromptLab Repository

PromptLab / STAT 4830 Repository Synthesis

May 13, 2026

Abstract

This report synthesizes the three main research threads implemented in the PromptLab repository: (1) a Pangram-based optimization tournament that compares multiple deslopification strategies under a shared evaluation harness, (2) a direct reinforcement-learning deslopifier trained with REINFORCE against EditLens, and (3) a two-stage "Promptlab-v2" pipeline that uses evolutionary prompt search to generate contrastive rewrite pairs and then distills them into a lightweight T5 rewriter. Taken together, these workstreams show that AI-detection evasion is indeed an optimization problem, but that the dominant challenge is not merely reducing a detector score. The harder problem is reducing that score while preserving semantics, length, fluency, and output validity. The tournament infrastructure demonstrates why a common benchmark is necessary: most entrants either worsened the detector score or appeared to improve only by violating similarity and length expectations. The RL deslopifier achieved the clearest EditLens-grounded gain, lowering mean EditLens from 0.999 to 0.670 and moving 45% of essays out of the fully-AI bucket, but it also exposed classic reward-hacking behavior on a subset of inputs. The Promptlab-v2 branch achieved the strongest detector-specific result, reducing mean slop from 0.391 to 0.0998 on held-out validation pairs, but under a different detector and with a clear T5 compression artifact. The overall lesson is therefore nuanced: detector-guided optimization works, yet its validity depends on the reward definition, the judge family, and the structural constraints imposed during search or training.

1 Introduction

The PromptLab repository studies what the team informally calls *AI slop*: text that is grammatical and often superficially competent, but still carries recognizable stylistic artifacts of machine generation. Rather than treating this only as a detection problem, the repository frames it as an optimization problem. The objective is to make text less detectably machine-generated, while constraining the rewrite process so that the result remains semantically faithful, fluent, and non-degenerate.

That framing produces an immediate tension. If the optimization target is only a detector score, the easiest solutions are often not the most meaningful ones. A model can shorten text, refuse to continue, repeat odd token patterns, or otherwise exploit detector blind spots without learning a human-like editing style. Much of the scientific value of this repository comes from confronting that tension directly. Across the codebase, detector-guided rewriting is explored through three different optimization paradigms:

1. a shared Pangram tournament that standardizes datasets, judges, and qualification gates;
2. online policy optimization with REINFORCE using EditLens as a continuous reward;

3. offline search-and-distillation, where an evolutionary prompt optimizer generates contrastive rewrite pairs for supervised training.

These workstreams do not share one perfectly unified metric regime. Early and late branches use different detectors, and some artifacts are directly comparable while others are only conceptually comparable. This report therefore treats the repository not as a single leaderboard, but as a coherent empirical program on detector-aware text optimization.

Before these three mature workstreams, the repository also contains a preliminary verifier stage. A Kaggle essay verifier notebook reports a DistilBERT-based classifier with test accuracy 0.993 and ROC-AUC 1.000 on its local task, while later experiment specifications explicitly note that such near-perfect performance was likely inflated by easy or dataset-specific artifacts. That early result is still important: it motivated the shift from simply classifying slop toward optimizing against it, and it foreshadowed the need for harder judges such as Pangram EditLens.

2 Scope and Evaluation Regimes

Any academic reading of the repository must begin with a measurement warning: the three main workstreams are not scored on one universal axis.

The Pangram tournament and the RL deslopifier both operate in the EditLens family, but even there the tournament uses a fast Pangram RoBERTa scorer during development while reserving Pangram 3B as the official judge. By contrast, the Promptlab-v2 branch is built around a local *SlopDetector* score and a separate search pipeline. This means that a number such as 0.0998 in Promptlab-v2 should not be read as directly superior to a number such as 0.670 in the RL pipeline. The former is a within-detector improvement on a small, curated validation set; the latter is an EditLens-grounded result on held-out Pangram data and a different score scale.

The repository is most coherent when viewed through common scientific questions rather than raw metric values:

- Can detector-aware optimization reduce detectability at all?
- What constraints are required to prevent reward hacking?
- Is online RL necessary, or can offline data generation and supervised learning recover the same effect?
- How much do conclusions depend on the judge family and evaluation harness?

3 Workstream I: The Pangram Optimization Tournament

3.1 Experimental motivation

The tournament is the repository's attempt to unify prior ideas under one controlled harness. The logic is explicit in the presentation materials and the implementation plan: earlier prompt hill-climbing showed that slop could be optimized against, and the RL deslopifier showed that direct detector reward could move outputs significantly, but neither alone established a stable comparative benchmark. The tournament addresses that gap.

Its design fixes the source pool, the base rewrite task, the Pangram judge family, and the qualification gates. Within that shared framework, the code compares several optimization strategies:

Workstream	Core method	Best result	Key caveat
Pangram tournament	Shared benchmark over LoRA, SFT, REINFORCE, and DPO variants under common similarity and length gates.	No robust winner; `M4` achieves only `+0.00035` mean delta in phase 2, with 16% pass rate and 0.598 median length ratio.	Once fidelity constraints are enforced, most apparent gains disappear.
RL deslopifier	Llama 3.2 3B Instruct with LoRA, optimized by REINFORCE against EditLens plus fluency and length regularization.	Mean EditLens falls `0.999 -> 0.670`; 45% of essays leave `LABEL_3` and 11% reach `LABEL_0`.	Some improvements come with reward hacking, refusals, or degenerate continuations.
Promptlab-v2	Evolutionary prompt search creates contrastive pairs that supervise a T5-base LoRA essay-to-essay rewriter.	Mean slop falls `0.391 -> 0.0998` on 20 held-out validation pairs.	Results are detector-specific and the T5 backbone tends to compress essay length.

Table 1: Repository workstreams and their main evaluation regimes.

attention-only LoRA (`M1`), full-module LoRA (`M2`), AdamW-supervised training (`M3`), REINFORCE (`M4`), curriculum SFT (`M5`), hard-negative SFT (`M6`), and DPO (`M7`).

This is a valuable scientific reframing. Instead of asking "did one clever model work?", the tournament asks which kind of optimization works under common constraints: broader parameterization, more stable first-order training, curriculum design, preference learning, or direct reward optimization.

3.2 Shared constraints and why they matter

The tournament's most important contribution may be methodological rather than purely performance-based. The implementation plan defines hard gates for semantic similarity, length ratio, and invalid-output rate. Those gates are not cosmetic. Without them, a deslopification system can appear to improve simply by becoming shorter, safer, or otherwise less essay-like.

The phase-2 leaderboard shows exactly why these gates matter. Across 250 evaluation rows, only `M4` posts a positive mean delta (`+0.000348`), yet it does so with only a 16% pass rate, mean similarity 0.849, and median length ratio 0.598. Every other method preserves semantics better, with similarity near 0.90 to 0.97, but worsens the detector score under the fast Pangram scorer, producing mean deltas between roughly `-0.029` and `-0.065`. Put differently: once the benchmark requires rewrites to remain recognizable as rewrites, the apparently easy optimization problem becomes genuinely hard.

This is an honest and scientifically useful negative result. It suggests at least three possibilities: the shared source pool is harder than the earlier detector-specific settings; the tournament entrants are still optimizing too shallowly, especially through aggressive shortening; and Pangram's evaluation family is less forgiving than the earlier local verifier regime.

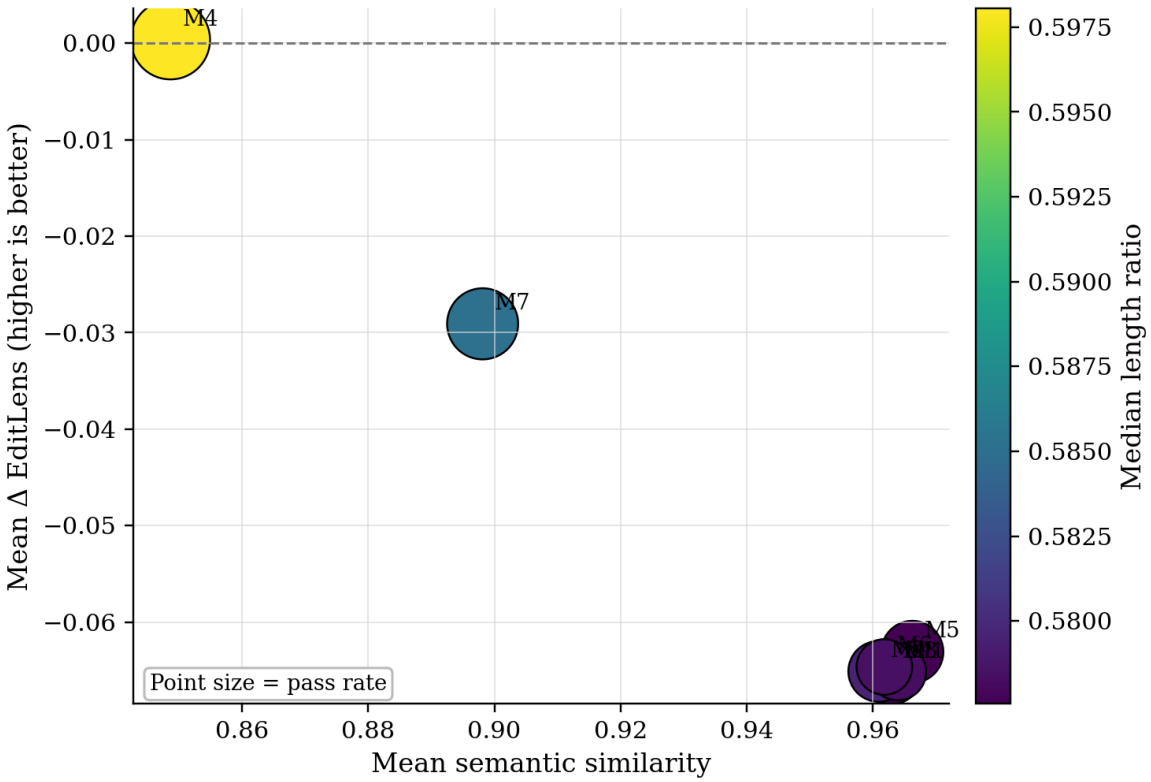

Figure 1: Phase-2 tournament tradeoff surface. Higher on the y-axis is better, but the only slightly positive run also pays a visible similarity and length cost.

3.3 Phase 1 and Phase 2 findings

The phase-1 results are similarly sobering. On 500 rows, all four initial entrants (`M1`–`M4`) show negative mean delta EditLens values, from approximately `-0.0064` to `-0.0125`, while median length ratios remain around 0.75. Phase 2 expands the entrant set and introduces pass-rate accounting, but the same pattern largely holds: no method establishes a robust Pareto-dominant operating point that both reduces Pangram score and clearly satisfies fidelity constraints.

The correct reading is not that the tournament failed. Rather, it performed its intended function. It showed that under a shared harness, many candidate methods that sound plausible on paper either do not improve the detector score or do so only by paying too much in similarity and length. For a repository devoted to optimization, that is exactly the kind of infrastructure result that prevents overclaiming.

4 Workstream II: Direct RL Deslopification with EditLens

4.1 Motivation and Objective Design

The RL deslopifier treats deslopification as a policy-gradient problem: given an AI-generated essay as a prompt, train a language model to produce rewrites that score lower on an AI detector while remaining fluent, coherent, and length-preserving. The choice of reward signal is the first and most consequential design decision. EditLens is not a binary classifier but an ordinal regression model producing a continuous score over four labeled buckets—fully human (LABEL_0, score

$\approx$ 0.007–0.035), lightly AI-edited (LABEL_1, 0.10–0.43), heavily edited (LABEL_2, 0.43–0.76), and fully AI-generated (LABEL_3, $\approx$ 0.999). This continuous structure is essential: a binary detector would provide almost no gradient signal early in training when all outputs score near 0.999, whereas EditLens yields a meaningful reward surface across the full training trajectory.

The Grammarly dataset analysis in the EditLens paper [6] establishes a further motivation: small, targeted edits—stronger verbs, varied sentence structure, naturalized dialogue—produce the largest score reductions, while wholesale rewrites often perform worse. This finding shapes the system prompt used throughout training:

> *You are a skilled human writing editor. Rewrite the following text to sound more like it was written by a thoughtful human. Make targeted, precise edits. Preserve the core meaning and approximate length.*

This framing steers the policy toward surgical editing rather than paraphrase, aligning the model's inductive bias with the reward landscape.

4.2 Pre-Training Validation (Experiments A1–A3)

Three validation experiments were completed before any RL training to confirm that the reward components were correctly implemented and scientifically justified.

A1 — EditLens probe. The script `scripts/probe_editlens.py` confirmed the canonical inference formula on real corpus data:

$$\hat{s} = \frac{1}{3} \sum_{k=0}^{3} k \cdot \text{softmax}(\mathbf{z})_k \tag{1}$$

where $\mathbf{z} \in \mathbb{R}^4$ are the raw logits from the four-class classifier head. The formula applies softmax followed by a weighted average divided by three—not a sigmoid, not raw logits. Empirical score ranges on real data were confirmed: human-written text scores 0.007–0.035, GPT-4.1-lightly-edited text scores approximately 0.347, and raw AI-generated text scores approximately 0.999. These baselines anchor the interpretation of all subsequent results.

A2 — Fluency complementarity. The fluency signal is defined as the log-probability of a candidate rewrite under a frozen copy of the Llama 3.2 3B base model—the same architecture used as the policy backbone. Text that the pre-trained base model considers likely is probably fluent, while degenerate or repetitive outputs receive very low probability under this reference. The experiment `scripts/fluency_reward_validation.py` tested whether this signal is complementary to EditLens or redundant with it. On a sample of held-out essays spanning human, AI-generated, and AI-edited categories, the Pearson correlation between EditLens score and raw fluency log-probability was $r = +0.303$ ($p = 0.032$) and Spearman $\rho = +0.458$ ($p < 0.001$). A correlation well below the $r > 0.8$ redundancy threshold confirms that both signals are doing independent work, justifying the fluency weight $\beta = 0.5$ in the combined reward.

A3 — Memory budget. The script `scripts/estimate_vram.py` confirmed that the full model stack—Llama 3.2 3B Instruct policy with LoRA, EditLens RoBERTa-large reward model, and frozen Llama 3.2 3B base as fluency reference—occupies approximately 14.9 GB VRAM under bfloat16 precision, well within the 80 GB A100 budget. This shared-reference design loads a single copy of the Llama 3.2 3B base on GPU to serve both the fluency scoring and the EditLens backbone, leaving 65 GB of headroom for the 48-rollout batch size.

4.3 Policy Model and LoRA Configuration

The policy model is Llama 3.2 3B Instruct [7], a decoder-only transformer pre-trained on a large text corpus and instruction-tuned to follow system and user prompts. Instruction tuning is important for this task: the base model assumption in REINFORCE requires the policy to already have non-negligible probability of producing reward-worthy outputs [2]. A base model with no instruction following cannot produce useful rewrites at initialization, making the gradient estimator degenerate.

Fine-tuning uses Low-Rank Adaptation (LoRA) [3] applied to the attention projections $\{q, k, v, o\}$ with rank $r = 16$ and scaling $\alpha = 32$. This yields approximately 9.2 million trainable parameters atop a 3.2 billion parameter frozen backbone (0.28%). LoRA provides three practical benefits: it keeps peak VRAM within budget, it prevents catastrophic forgetting of the base model's language prior, and it allows the LoRA adapter to be saved and loaded independently of the 6 GB base weights for checkpoint management.

4.4 Reward Function

The combined reward for a rewrite Y given input essay X is:

$$R(Y) = \alpha \cdot (1 - \mathrm{EL}(Y)) + \beta \cdot \tilde{F}(Y) + \gamma \cdot L(Y) \tag{2}$$

where:

- $\mathrm{EL}(Y) \in [0, 1]$ is the EditLens score; the detection-evasion term $(1 - \mathrm{EL}(Y))$ rewards lower AI-detectability with weight $\alpha = 1.0$;
- $\tilde{F}(Y)$ is the fluency score z-normalized across the batch before reward computation, defined as the per-token log-probability of Y under the frozen Llama 3.2 3B base model, with weight $\beta = 0.5$ (validated by A2);
- $L(Y)$ is a length-match reward equal to 1.0 if the output length ratio $|Y|/|X|$ falls in $[0.40, 1.60]$ and decays linearly outside that range, with weight $\gamma = 0.1$.

A minimum-length filter rejects any rewrite under 50 tokens or under 40% of the input token count before reward computation. This guards against the trivial exploit of producing very short outputs that score well on EditLens without genuinely rewriting the essay.

4.5 Training Algorithm and Infrastructure

Training follows the REINFORCE policy gradient estimator [2]:

$$\nabla_\theta F(\theta) = \mathbb{E}_{Y \sim p_\theta(\cdot|X)}[R(Y) \cdot \nabla_\theta \log p_\theta(Y \mid X)] \tag{3}$$

approximated by a batch of 48 rollouts per step.

Advantage normalization and baseline evolution. When all rewards are positive—as they are here, with values in the range 0.06–0.15—vanilla REINFORCE can only increase probabilities, never decrease them. Subtracting a baseline b from each reward fixes this: the expected gradient is unchanged since $\mathbb{E}[\nabla_\theta \log p_\theta] = 0$, but samples below the baseline receive negative weight and are explicitly suppressed.

A preliminary training run used an exponential moving average (EMA) of past rewards as the baseline. This required warmup: for the first 50–150 steps the EMA was poorly estimated, producing noisy gradients and visible metric regression in that run (mean EditLens increased from

0.752 at step 50 to 0.852 at step 150 before recovering). The production C2 training run switched to *batch advantage normalization*, subtracting the batch mean and dividing by the batch standard deviation within each step's 48 rollouts:

$$A_i = \frac{R(Y_i) - \bar{R}}{\sigma_R + \epsilon}, \quad \bar{R} = \frac{1}{48}\sum_j R(Y_j), \quad \sigma_R = \mathrm{std}(\{R(Y_j)\}) \tag{4}$$

This is zero-warmup—roughly half the batch receives positive weight and half receives negative weight from step one—and is structurally equivalent to GRPO [8] with batch-level rather than within-group normalization. The switch produced monotonic improvement in the C2 run with no early oscillation.

Batched generation. An initial implementation generated rollouts sequentially in a Python loop. At 48 rollouts with up to 512 output tokens each, this required approximately 15 minutes per training step—completely infeasible for 500 steps. The production implementation replaces the sequential loop with a single batched `model.generate()` call over all 48 left-padded prompts, reducing per-step time to approximately 50 seconds on an A100 80 GB.

Memory management in Phase 3. The REINFORCE backward pass originally accumulated gradients across all 48 rollouts before calling `loss.backward()`, holding all intermediate activations in VRAM simultaneously. With a 3B policy model this caused out-of-memory (OOM) failures. The fix processes one rollout at a time with immediate `.backward()` followed by `torch.cuda.empty_cache()`, dividing each per-rollout loss by the number of accepted rollouts to preserve the correct gradient scale. This is mathematically equivalent to the mean loss backward but keeps peak VRAM to one rollout's activations at a time.

Smoke test (C1). Before the 500-step production run, a 10-step smoke test using Qwen 0.5B as the policy and 5 training essays confirmed the full pipeline executed correctly: finite non-NaN rewards, gradient norms in $[10^{-4}, 10]$, LoRA parameters changed from initialization, and the minimum-length filter firing correctly. All seven checklist items passed.

Hyperparameters. The locked configuration: AdamW optimizer, lr = 10^{-5}, weight decay 0.01, gradient clip norm 1.0, KL penalty weight 0.1 (subtracted from reward per rollout), 500 steps, 48 rollouts per step, eval every 50 steps on 100 held-out essays (50 `ai_generated`, 50 `ai_edited` from the Pangram EditLens test split), checkpoint every 100 steps. Both input essays and generated rewrites were capped at 512 tokens, consistent with the EditLens RoBERTa reward model's maximum context length. The training pool consisted of 1,000 randomly sampled `ai_generated` essays from the Pangram EditLens training split, with 48 fresh essays sampled per step.

4.6 Main Quantitative Results

The evaluation log in `outputs/c2_eval.jsonl` records the following trajectory across 500 training steps:

Mean EditLens falls from 0.999 to 0.670 over 500 steps, a reduction of 0.329 absolute points. At the label level, 45% of essays exit the fully-AI bucket: 34% move to LABEL_1 (lightly edited) and 11% reach LABEL_0 (fully human range). KL divergence increases gradually and stabilizes at 0.518 at step 500, remaining in the well-behaved training regime throughout. The oscillation visible around steps 300–350, where mean EditLens briefly reverses before recovering, is consistent with REINFORCE's characteristically high gradient variance rather than a training instability.

A notable observation is that zero essays land in LABEL_2 (heavily edited, 0.43–0.76) at most checkpoints. This is consistent with the continuous reward gradient pushing essays through the transitional zone rather than settling there: with a monotone reward, LABEL_2 is a waypoint rather than a stable attractor.

Step	Mean EditLens	LABEL_0%	LABEL_1%	LABEL_3%	KL div
0 (base)	0.999	0.0	0.0	100.0	0.000
50	0.768	1.0	29.0	70.0	0.397
100	0.766	0.0	32.0	68.0	0.422
150	0.754	0.0	36.0	64.0	0.457
200	0.725	3.0	39.0	58.0	0.481
250	0.682	4.0	44.0	52.0	0.513
300	0.703	5.0	35.0	60.0	0.550
350	0.718	5.0	33.0	62.0	0.551
400	0.675	11.0	31.0	58.0	0.580
450	0.685	9.0	34.0	56.0	0.545
500	0.670	11.0	34.0	55.0	0.518
Δ (0→500)	−0.329	+11.0	+34.0	−45.0	+0.518

Table 2: C2 training progression from `outputs/c2_eval.jsonl`. Evaluation on 100 held-out essays every 50 steps.

4.7 KL Penalty Ablation (Experiments C4-a and C4-c)

To understand the role of the KL regularization term, two additional 500-step runs were conducted varying only the KL penalty weight while holding all other hyperparameters fixed:

Run	KL Penalty	Mean EditLens	LABEL_0%	KL div
C4-a	0.0	0.633	12	0.637
C2	0.1	0.670	11	0.518
C4-c	0.5	0.431	48	3.735

Table 3: KL penalty ablation at step 500. Results from `outputs/c4a_eval.jsonl` and `outputs/c4c_eval.jsonl`.

C4-a (no KL penalty) produces results marginally better than C2 on detector score (0.633 vs. 0.670) with only slightly higher KL divergence (0.637 vs. 0.518). This suggests that batch advantage normalization provides sufficient regularization at this scale and the explicit KL penalty in C2 was mildly over-constraining.

C4-c (KL penalty 0.5) appears dramatically better on the detector score alone (0.431, 48% LABEL_0). However, its KL divergence of 3.735 is 7× higher than C2's 0.518, indicating the model drifted far outside the well-behaved training regime. Qualitative inspection confirms the same degenerate patterns documented in E3: partial rewrites followed by repetitive token fragments that reduce the EditLens score without producing readable text. The project interprets C4-c as a reward-hacking condition rather than genuine improvement.

This finding directly mirrors the pattern documented in the course's reward-shaping experiment [9]: the KL penalty consistently changed which degenerate mode the model collapsed to without preventing collapse. The key diagnostic insight is that *KL divergence above approximately 1.0 serves as a reliable signal for reward hacking.* When KL remains low (as in C2 and C4-a), EditLens

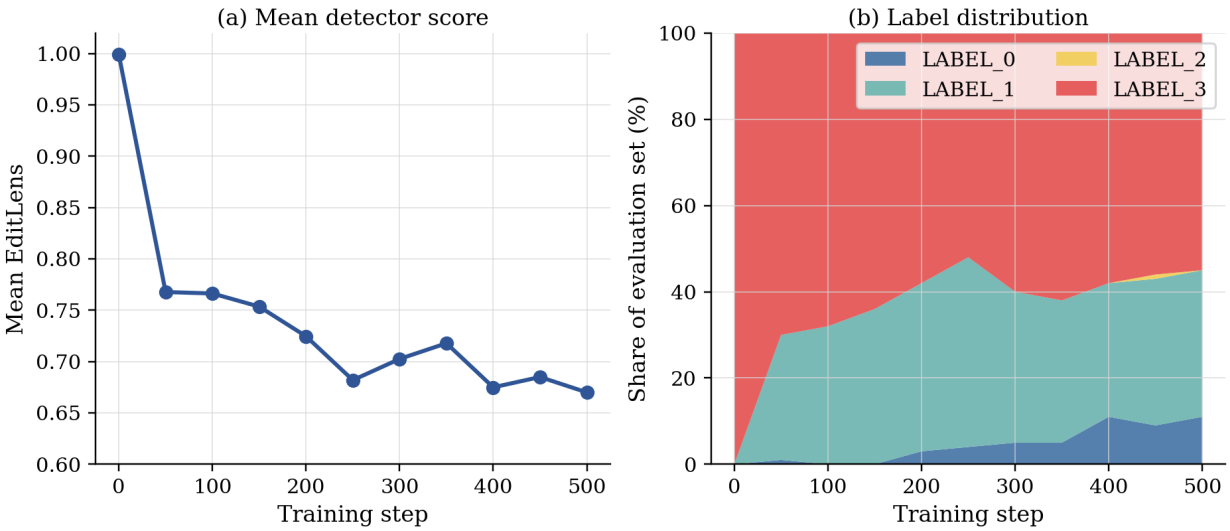

Figure 2: C2 training progression from `outputs/c2_eval.jsonl`. Left: mean EditLens over 500 training steps. Right: label-mass transfer, showing 45% of essays exiting LABEL_3 by step 500.

improvements correspond to genuine linguistic changes. When KL explodes (as in C4-c), the apparent gains are artifacts of distributional collapse.

4.8 Qualitative Behavior (Experiment E3)

A qualitative evaluation loaded the step-500 checkpoint and generated rewrites for 20 `ai_generated` essays from the test split using temperature sampling ($T = 0.8$). Three distinct behavioral patterns emerged across the 20 essays.

Genuine targeted editing (approximately 4–6 of 20 essays). On journalistic and narrative essays, the model made surgical linguistic improvements consistent with the edit categories identified in the EditLens Grammarly analysis: replacing weak existential verbs with vivid action verbs, adding sensory grounding, restructuring sentences for tension, and naturalizing dialogue. For example, Essay 2 (Liberation of Paris) changed:

> *Before:* "Paris in the summer of 1944 was a city on the edge."
> *After:* "In the sweltering summer of 1944, Paris teetered on the precipice of collapse."

Essay 16 (detective fiction) changed:

> *Before:* "I know you're in here, Martinez," he called out.
> *After:* "Martinez, is that you?" he called out.

The detective passage improved from EditLens score 0.9922 to 0.5943. The model discovered these targeted edit strategies from the reward signal alone without being programmed to apply them. This is noteworthy because the same edit categories identified by the EditLens Grammarly analysis were recovered through optimization, suggesting the reward surface implicitly encodes linguistically meaningful gradients.

Reward hacking (approximately 6 of 20 essays). Several essays produced large EditLens score reductions paired with degenerate output: partial rewrites followed by repetitive token

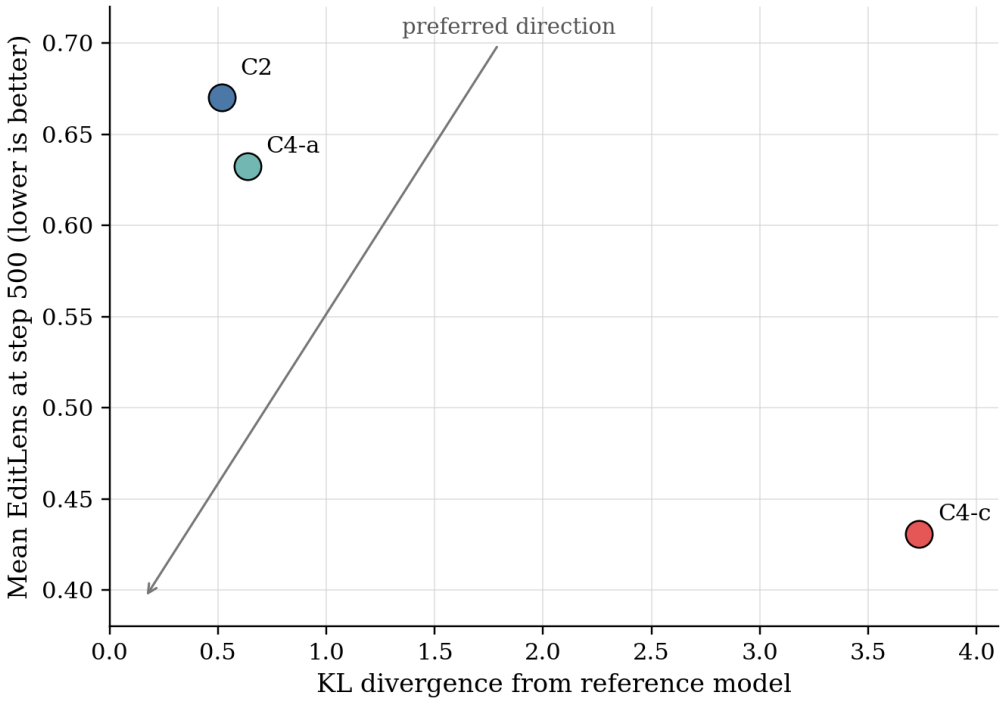

Figure 3: KL penalty ablation at step 500. Detection evasion (x-axis, higher is better) plotted against KL divergence (y-axis, lower is more constrained). C2 and C4-a form the valid operating region; C4-c's large KL marks an implausible operating point consistent with reward hacking.

fragments, or safety refusals triggered by the model's instruction-tuning guardrails in response to content it flagged as problematic. These cases reduce the detector score without constituting genuine rewrites.

No change (approximately 8 of 20 essays). A substantial fraction of essays showed minimal EditLens movement and outputs nearly identical to the input. These appear concentrated in content categories where the model's generation style does not naturally produce human-like variation.

The qualitative result is consistent with the aggregate eval trajectory. The 34% of essays reaching LABEL_1 is more plausibly explained by genuine learning, while the 11% reaching LABEL_0 is more consistent with reward hacking given the magnitude of the required score reduction.

4.9 Multi-Detector Evaluation (Experiment E1)

To assess cross-detector generalization, the step-500 checkpoint was evaluated on 20 held-out essays using two independent signals: EditLens RoBERTa-large and perplexity under the frozen Llama 3.2 3B base model. Higher perplexity indicates text that is more surprising to the language model, which can correlate with human-like writing that does not follow predictable AI patterns.

Metric	Original	Rewrite
Mean EditLens	0.990	0.817
Mean perplexity	10.94	57.10

Table 4: E1 multi-detector results from `outputs/e1_summary.txt` across 20 held-out essays.

The EditLens improvement (0.990 → 0.817) is consistent with the held-out eval results. The perplexity increase (10.94 → 57.10), however, is dominated by two outlier essays with perplexity values of 152 and 862—outputs that are highly unpredictable because they are degenerate rather than human-like. Excluding these outliers, perplexity slightly decreases for the remaining essays, meaning the genuine rewrites are more fluent and coherent than the originals while the reward-hacked outputs are less so. This pattern suggests partial cross-detector transfer: genuine rewrites generalize to the perplexity signal, while reward-hacked outputs do not. Full generalization would require training against multiple detectors simultaneously.

4.10 What Was Not Completed

The experiment specification included a GRPO comparison run (C3), a reward weight ablation (C5), a model size sweep (D1), and a rewriter-vs-generator architectural comparison (D2). The C3 GRPO script was written and launched but did not produce saved outputs before the final presentation deadline. C5, D1, and D2 were not started due to compute time constraints. The most scientifically important missing experiment is C5, which would test a detection-only reward ($\beta = 0$) as a control condition, allowing precise attribution of how much of the EditLens improvement is attributable to the fluency regularizer versus the detection evasion term alone.

4.11 Summary

The RL deslopifier demonstrates that policy-gradient optimization against a continuous AI detector produces meaningful and measurable improvements on held-out data. The main quantitative finding—mean EditLens 0.999 → 0.670, with 45% of essays exiting the fully-AI zone—is the strongest external-detector result in the repository. The qualitative and ablation analyses reveal that genuine linguistic learning and reward hacking coexist within the same trained model, and that KL divergence above approximately 1.0 provides a reliable diagnostic for distinguishing between them. The most important open question is whether these EditLens-grounded improvements transfer to human judgment—a question that can only be answered by human evaluation, which remains as future work.

5 Workstream III: Promptlab-v2 Evolutionary Search and Supervised Distillation

5.1 Stage 1: evolutionary prompt search

The Promptlab-v2 branch takes a different view of the same core problem. Instead of optimizing a rewrite model directly against a detector at inference time, it first uses the detector as a search signal to generate training data.

The stage-1 pipeline maintains a population of prompt candidates and repeatedly mutates and recombines them. A small mutator model (`llama-3.1-8b-instant`) proposes variants; a larger model (`llama-3.3-70b-versatile` through the Groq API) generates essays from them; and a detector-based objective selects the best outputs. Two design choices are especially important:

- a drift penalty, which penalizes semantic drift, overlap failure, and poor fluency so that the optimizer cannot win by producing incoherent detector-evading text;
- few-shot injection, where successful low-slop essays are recycled as exemplars in later rounds, allowing the search process to bootstrap its own stylistic prior.

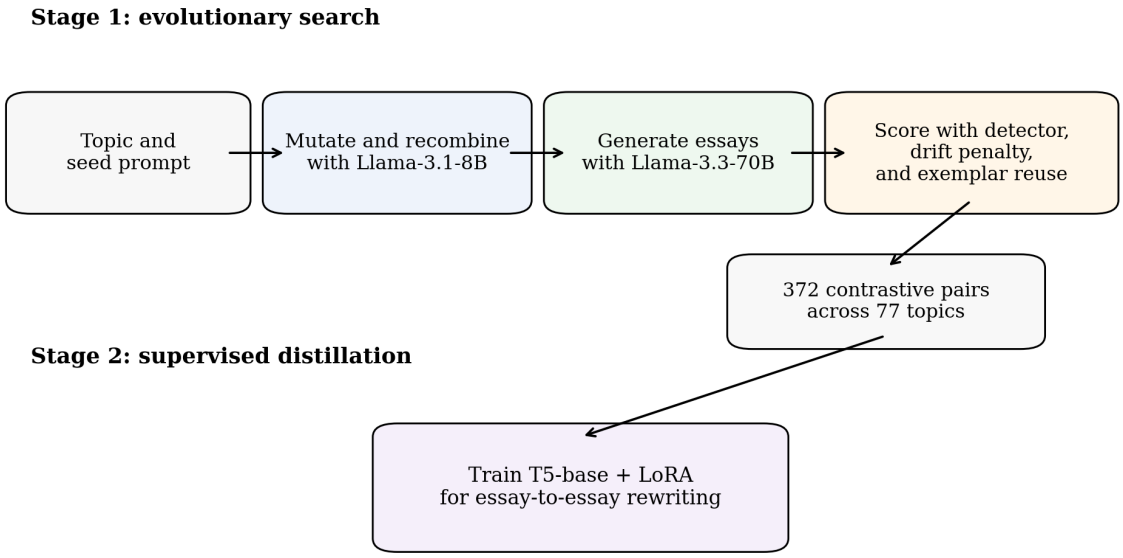

Figure 4: Reconstructed Promptlab-v2 pipeline based on repository code paths and presentation notes. The detector is used first as a search signal and then indirectly through supervised distillation.

According to the presentation materials, this stage yields 372 co-training pairs over 77 topics, from which a smaller subset of high-contrast pairs is used for supervised training.

5.2 Stage 2: essay-to-essay supervised distillation

Stage 2 turns those search results into a direct rewriter. The model is T5-base with a LoRA adapter, with the speaker notes reporting roughly 800,000 trainable parameters on top of a 220M-parameter backbone. The key architectural change is a shift from prompt-to-prompt optimization to essay-to-essay rewriting. Instead of learning which prompts might later generate low-slop essays, the model learns to map a high-slop essay directly to a lower-slop essay.

Several training decisions appear to matter:

- AdamW with decoupled weight decay (`0.01`);
- cosine learning-rate decay from `1e-4` with 50 warmup steps;
- curriculum ordering by slop gap, so the model sees the most contrastive pairs first;
- early stopping on the task metric (`slop_mean`) rather than pure cross-entropy;
- deterministic beam search during evaluation for stable checkpoint comparison.

This branch is conceptually important because it offers an offline alternative to direct online reward optimization. The expensive search happens once, and the resulting rewrite model can then run in a single forward pass.

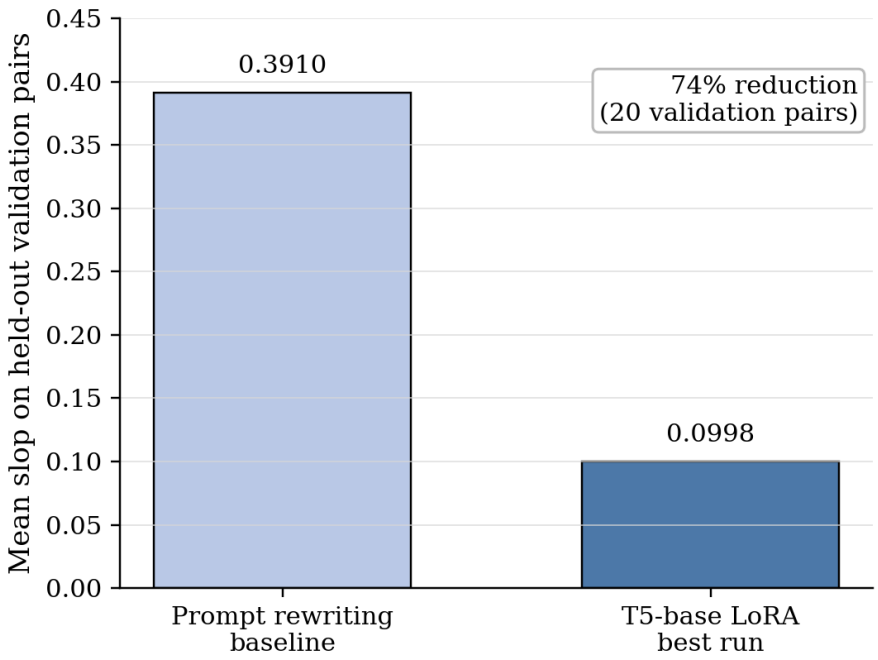

Figure 5: Promptlab-v2 validation result summarized from the project notes: a 74% reduction in mean slop relative to the prompt-rewriting baseline on 20 held-out validation pairs.

5.3 Results and limitations

The final Promptlab-v2 model is the strongest detector-specific performer reported in the repository. On 20 held-out validation pairs, the best configuration achieves mean slop `0.0998`, compared with a `0.391` prompt-rewriting baseline, for a 74% reduction. The best run uses `LR = 1e-4`, `r = 16`, and `weight_decay = 0.01`; an `r = 8` ablation reportedly finishes only `0.006` behind.

These gains come with two critical caveats. First, the dataset is extremely small. The stage-2 best run is ultimately trained on only 40 training pairs with 20 validation pairs, which the presentation explicitly identifies as the binding constraint. The reported "step-50 noise floor" is consistent with a high-variance small-data regime. Second, the model is visibly biased toward summarization. The example-essay analysis in the presentation notes indicates that the T5 system often achieves low detector scores by compressing the essay into a shorter text, not by preserving full essay length and rhetorical breadth. That is not a trivial flaw: if the user task is full-length rewriting, then compression is a form of partial task failure even when detector evasion improves.

This branch therefore provides a compelling proof of concept for offline data-generation and distillation, but not yet a detector-agnostic final answer.

6 Comparative Discussion

6.1 What the three workstreams jointly show

The repository's main scientific lesson is that deslopification is real but brittle. Each workstream contributes a different piece:

- The tournament shows that common constraints are necessary, because otherwise detector-aware systems can win for the wrong reasons.

- The RL branch shows that direct online reward optimization can move a strong detector by a large amount, but it is vulnerable to reward hacking.
- The Promptlab-v2 branch shows that expensive search can be distilled into a lightweight supervised model, but only under a detector-specific regime and with substantial output-length bias.

Across all three, the theme is the same: raw optimization power is not enough. Objective design, regularization, and evaluation discipline determine whether a lower detector score corresponds to a better rewrite.

6.2 Why the workstreams are not numerically interchangeable

It would be tempting to read the Promptlab-v2 result (`0.0998`) as dominating the RL result (`0.670`). That conclusion would be incorrect. The branches differ in detector family, data regime, and validation structure. Promptlab-v2 is best interpreted as showing that offline search plus distillation can be highly effective on its chosen detector and curated pair set. The RL branch, by contrast, is the strongest evidence that large-scale end-to-end detector reward can move a modern external judge on held-out essays. The tournament sits between them as an attempt to standardize that comparison, and its negative results should be understood as a warning that cross-method evaluation is harder than it looks.

6.3 Ethics and responsible framing

This repository studies detector evasion, which has obvious misuse potential. The strongest academically defensible framing is not "how to bypass detectors in production," but rather "how optimization behaves against imperfect reward models and what failure modes that exposes." On that framing, the project is valuable precisely because it shows how easy it is to overfit a detector and how necessary human and multi-detector evaluation are before making claims about genuine writing quality.

7 Conclusion

The PromptLab repository contains a coherent body of work on optimization against AI detectability, but it does not support a simplistic victory narrative. The Pangram tournament contributes a rigorous shared harness and exposes how many apparent improvements disappear under fidelity constraints. The REINFORCE deslopifier provides the clearest external-detector result, moving mean EditLens from 0.999 to 0.670 and shifting nearly half the evaluation set out of the fully-AI region, while simultaneously documenting concrete reward-hacking cases. The Promptlab-v2 branch demonstrates that detector-guided search can be distilled into a lightweight essay-to-essay model with very strong detector-specific gains, though on a different judge and with clear summarization bias.

The most defensible synthesis is therefore this: AI-detection evasion is an optimization problem with real signal, but the scientific bottleneck is objective validity, not optimization capability alone. Future progress should unify these workstreams under common judges, larger contrastive datasets, stronger semantic constraints, and human evaluation.

References

[1] K. Thai, B. Emi, E. Masrour, and M. Iyyer. EditLens: Quantifying the Extent of AI Editing in Text. ICLR 2026 / arXiv:2510.03154.

[2] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 1992.

[3] E. Hu, Y. Shen, P. Wallis, et al. LoRA: Low-Rank Adaptation of Large Language Models. 2021.

[4] C. Raffel, N. Shazeer, A. Roberts, et al. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *JMLR*, 2020.

[5] R. Rafailov, A. Sharma, E. Mitchell, et al. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. 2023.

[6] K. Thai, B. Emi, F. Masrour, and M. Iyyer, "EditLens: Quantifying the Extent of AI Editing in Text," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.03154.

[7] Meta AI, "The Llama 3 Herd of Models," 2024. `https://ai.meta.com/research/publications/the-llama-3-herd-of-models/`

[8] Z. Shao, P. Wang, Q. Zhu, R. Xu, Y. Song, M. Zhang, Y.K. Li, Y. Wu, and D. Guo, "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," *arXiv preprint arXiv:2402.03300*, 2024.

[9] D. Davis, "STAT 4830: Reinforcement Learning for Language Models," Course notes, University of Pennsylvania, 2025. `https://damek.github.io/STAT-4830/section/7/notes.html`